

Day3. 객체지향 설계

1. 수업내용 정리

1.1 객체 생성패턴

- 생성자 패턴: 필드 2~3개, 단순한 경우
 - 장점: 직관적, 빠름 / 단점: 매개변수 많으면 가독성 저하
- 정적 팩토리 메소드: 생성 의미를 명확히 할 때
 - 장점: 이름으로 의도 표현, 캐싱 가능 / 단점: 상속 어려움
- 빌더 패턴: 필드 많고 선택적인 경우
 - 장점: 가독성 최고, 선택 필드 처리 용이 / 단점: 코드량 증가

Q. 왜 패턴이 필요한가?

객체 생성 방식이 복잡해지면 코드 중복, 의존성 증가, 테스트 어려움등의 문제점이 발생한다.
그래서, 자주 사용하는 코드 패턴을 정리해서 관리한다.

대표 패턴

- 싱글톤(Singleton): 객체를 하나만 생성

```
class Singleton {
    private static final Singleton instance = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return instance;
    }
}
```

- 팩토리 패턴(Factory): 객체 생성 로직을 한 곳에 모아서 관리하는 패턴, new 여기저기 직접 사용하지 않고 한 곳에서 관리

```
// 공통 타입
public interface User {
    void printRole();
}

// 구현 클래스
public class AdminUser implements User {
    public void printRole() {
        System.out.println("관리자입니다.");
    }
}

public class NormalUser implements User {
    public void printRole() {
        System.out.println("일반 사용자입니다.");
    }
}

// 팩토리 패턴
public class UserFactory {
    public static User create(String type) {
        if ("admin".equals(type)) {
            return new AdminUser();
        } else if ("user".equals(type)) {
            return new NormalUser();
        }
        throw new IllegalArgumentException("알 수 없는 사용자 타입");
    }
}

// 실제 사용 방법
```

```
User user = UserFactory.create("admin");
user.printRole();
```

- 빌더 패턴(Builder): 필드가 많거나 선택적 값이 많은 객체를 안전하게 생성하기 위한 패턴

```
public class User {
    // 캡슐화 및 final: 외부 접근 불가, 객체 생성 이후 값 변경 불가
    private final String id;
    private final String password;
    private final String role;
    private final boolean active;

    // Builder를 통해서만 객체를 생성하도록 private으로 생성자 강제
    private User(Builder builder) {
        this.id = builder.id;
        this.password = builder.password;
        this.role = builder.role;
        this.active = builder.active;
    }

    // 내부 정적 클래스로 User객체 없이도 Builder를 만들 수 있음.
    public static class Builder {
        private String id;
        private String password;
        private String role = "USER"; // 기본값
        private boolean active = true; // 기본값

        // 메서드 체이닝
        public Builder id(String id) {
            this.id = id;
            return this;
        }

        // 해당 메서드 호출로만 User 객체 생성
        public User build() {
            return new User(this);
        }
    }

    // 사용 방법
    User user = new User.Builder()
        .id("admin")
        .password("1234")
        .role("ADMIN")
        .build();
}
```

1.2 SOLID 원칙과 SpringBoot 적용 흐름

약자	원칙(영문)	원칙(한글)	핵심 내용	Spring Boot와의 연관성
S(SRP)	Single Responsibility Principle	단일 책임 원칙	하나의 클래스는 하나의 책임만 가져야 한다.	Controller, Service, Repository 등 계층 분리
O(OCP)	Open-Closed Principle	개방-폐쇄 원칙	기존 코드는 수정하지 않고 기능을 확장할 수 있어야 한다.	인터페이스 기반 구현체 추가, 전략 패턴 활용
L(LSP)	Liskov Substitution Principle	리스코프 치환 원칙	부모 타입 대신 자식 객체를 사용해도 동일하게 동작해야 한다.	인터페이스 구현체 교체 가능
I(ISP)	Interface Segregation Principle	인터페이스 분리 원칙	필요한 기능만 가진 작은 인터페이스로 분리한다.	역할별 인터페이스 설계
D(DIP)	Dependency Inversion Principle	의존성 역전 원칙	구현 클래스가 아닌 추상화(인터페이스)에 의존한다.	Spring의 DI와 IoC의 핵심 원칙

1.3 OOAD 정리

절차기반

- 코드를 위에서 아래로 순서대로 실행
- 함수 중심 - 데이터와 로직이 분리
- 수정 시 전체 흐름에 영향

객체지향

- 현실 세계를 객체(데이터+행위)로 모델링
- 클래스 중심 - 책임이 분산
- 한 클래스 수정이 다른 클래스에 영향 최소화

1.4 Spring Boot 개요

Spring Framework

- 설정파일 직접 작성
- DI, AOP, MVC 등 핵심 기능 제공
- 유연하지만 초기 설정 많음
- 톰캣 등 서버 별도 설치 필요

Spring Boot

- Auto Configuration - 설정 자동화
- 내장 톰캣 포함 - JAR 하나로 실행
- spring-boot-starter-* 의존성 묶음
- 개발자가 비즈니스 로직에만 집중 가능

2. 코드 분석

2.1. main

기존 main 코드의 문제점:

1. 객체의 생성을 main 코드 내부에서 진행
2. GET/POST API 처리도 내부에서 진행

Spring Boot 적용 후:

```
package loginauth;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LoginAuthApplication {
    public static void main(String[] args) {
        SpringApplication.run(LoginAuthApplication.class, args);
    }
}
```

- main 실행 코드가 현저히 짧아진 것을 확인할 수 있음.
- 스프링부트 프로젝트는 레이어별 패키지 구조로 작성.
- 현재 코드 구조는 다음과 같음.

Controller(또는 Web) - HTTP 요청을 받는 계층
Service - 비즈니스 로직 계층
Repository(또는 DAO) - DB 접근 계층
Domain(또는 Entity, Model) - @Entity 같은 도메인 객체
DTO - 계층 간 데이터 전달용 객체

Config - 설정 클래스
Exception - 예외 및 예외 처리기
Security - 인증/인가 관련 설정

Q. Bean은 무엇인가?

Spring IoC 컨테이너가 직접 생성하고, 의존성을 주입하며, 생명주기를 관리하는 자바 객체 (싱글톤으로 객체 생성)

객체를 Bean으로 등록하는 방법

1. 어노테이션 이용
2. @Configuration 작성 - 외부 라이브러리 객체처럼 직접 소스코드를 수정할 수 없는 클래스를 빈으로 등록할 때 사용

2.2. DB 조작

```
// 변경 전
public interface UserRepository {
    void save(User user) throws Exception;
    User findByUsername(String username) throws Exception;
}

public static class JdbcUserRepository implements UserRepository {
    private final String url;
    private final String user;
    private final String password;

    // 생성자(constructor): new JdbcUserRepository(...)로 객체를 만들 때 딱 한 번 실행되며,
    // 필드 초기화를 담당합니다. 메소드처럼 보이지만 이름이 클래스명과 같고 반환 타입이 없습니다.
    public JdbcUserRepository(String url, String user, String password) {
        this.url = url;
        this.user = user;
        this.password = password;
    }

    ...
}

// 변경 후
// UserRepository.java
public interface UserRepository {
    User save(String username, String encodedPassword);
    Optional<User> findByUsername(String username);
}

// JdbcUserRepository.java
public class JdbcUserRepository implements UserRepository {
    private final JdbcTemplate jdbcTemplate;

    public JdbcUserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    ...
}
```

- 변경 전 코드에서는 직접 생성자에서 url, user, password를 입력 받아 저장하는 방식을 사용함.
- 변경 후 코드는 Spring Framework에서 제공하는 JDBC 템플릿을 입력 받아 저장함.
- 이 템플릿에서는 DB를 다룰 때 필요한 명령들을 쉽게 메서드로 제공함.
- 인터페이스 또한 오류 반환에서 User 객체를 반환하는 방식으로 변경되었다.
- Optional 키워드는 null을 반환할 수 있다는 키워드이다.

2.3. User

```
// 변경 전
public static class User {
    private Long id;          // Long: 기본형 long의 "객체 버전"(래퍼 클래스). null을 가질 수 있어 "아직 없음"을
    표현 가능
    private String username;
    private String password; // 주의: 여기 저장되는 값은 평문이 아니라 BCrypt 해시값 (아래 AuthService 참고)

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; } // this.id: "필드 id"를 가리킴 (매개변수 id와 이름이 같아
    구분 필요)

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}

// 변경 후
public record User(Long id, String username, String password) {
}
```

- 변경 전 코드에서는 캡슐화를 직접 진행하고 static 처리를 해서 객체화를 시켰다.
- 그러나 변경 후 코드는 단 한줄의 코드를 통해 위 코드에서 진행하던 로직을 처리하고 있다.
- **record** 키워드는 Java16 부터 추가된 키워드로 컴파일 시 getter, toString, equals 등 함수를 자동으로 생성해준다.
- 불변 데이터 전달용으로 사용되는 클래스임으로 생성 후 값을 변경할 수는 없다.

2.4. Service

```
// 변경 전
public static class AuthService {
    private final UserRepository userRepository;

    public AuthService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // 회원가입
    public void signUp(String username, String rawPassword) throws Exception {
        User existing = userRepository.findByUsername(username); // ↑
        JdbcUserRepository.findByUsername 호출
        if (existing != null) {
            throw new SQLIntegrityConstraintViolationException("username already exists");
        }

        String hashed = BCrypt.hashpw(rawPassword, BCrypt.gensalt(12));

        User u = new User();
        u.setUsername(username);
        u.setPassword(hashed); // 평문이 아니라 해시값을 저장 -> DB가 털려도 원문 비밀번호는 알 수 없음
        userRepository.save(u); // ↑ JdbcUserRepository.save 호출
    }

    public User login(String username, String rawPassword) throws Exception {
        User u = userRepository.findByUsername(username);
        if (u == null) {
            return null; // 그런 사용자 자체가 없음
        }

        if (!BCrypt.checkpw(rawPassword, u.getPassword())) {
            return null; // 비밀번호 불일치
        }

        return u; // 로그인 성공 -> 이 User 객체가 호출부(handler)로 돌아가 세션/JWT 발급에 쓰임
    }
}
```

```

}

// 변경 후
@Service
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Transactional
    public User signUp(String username, String rawPassword) {
        userRepository.findByUsername(username).ifPresent(user -> {
            throw new DuplicateUsernameException(username);
        });
        return userRepository.save(username, passwordEncoder.encode(rawPassword));
    }

    @Transactional(readOnly = true)
    public User authenticate(String username, String rawPassword) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(InvalidCredentialsException::new);
        if (!passwordEncoder.matches(rawPassword, user.password())) {
            throw new InvalidCredentialsException();
        }
        return user;
    }
}

```

- 변경 전 AuthService에서는 직접 객체를 생성하고, 암호화 역시 직접 호출해서 처리함.
- 또한 DB 조회 결과가 없으면 null을 반환하기 때문에, 호출하는 쪽에서 매번 null 체크가 필요함.
- 변경 후에는 @Service 어노테이션을 붙여 Bean으로 등록하고, 패스워드 인코더와 유저레퍼지토리를 주입함.
- @Transactional을 사용해서 로직이 하나의 트랜잭션 안에서 실행되도록 함.
- 실패 상황에서는 null 대신 직접 작성한 예외 코드를 반환함.

2.5. Controller

```

// AuthController.java
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;
    private final JwtProvider jwtProvider;
    private final JwtProperties jwtProperties;
    private final CookieService cookieService;

    public AuthController(
        AuthService authService,
        JwtProvider jwtProvider,
        JwtProperties jwtProperties,
        CookieService cookieService
    ) {
        this.authService = authService;
        this.jwtProvider = jwtProvider;
        this.jwtProperties = jwtProperties;
        this.cookieService = cookieService;
    }

    /*
    * 회원가입
    *
    * signup.html이 일반 form 방식으로
    */
}

```

```

    * application/x-www-form-urlencoded 요청을 보내므로
    * @RequestBody는 사용하지 않습니다.
    */
@PostMapping("/signup")
public void signup(
    @Valid LoginRequest request,
    HttpServletResponse response
) throws IOException {

    authService.signUp(
        request.username(),
        request.password()
    );

    response.sendRedirect("/login");
}

/*
 * 로그인
 */
@PostMapping("/login")
public void login(
    @Valid LoginRequest request,
    HttpServletResponse response
) throws IOException {

    User user = authService.authenticate(
        request.username(),
        request.password()
    );

    String token = jwtProvider.createAccessToken(user);

    response.addHeader(
        HttpHeaders.SET_COOKIE,
        cookieService
            .accessToken(
                token,
                jwtProperties.accessTokenTtl()
            )
            .toString()
    );

    response.addHeader(
        HttpHeaders.SET_COOKIE,
        cookieService
            .appAuth(
                jwtProperties.accessTokenTtl()
            )
            .toString()
    );

    response.sendRedirect("/home.html");
}

/*
 * 현재 로그인 사용자 인증 확인
 */
@GetMapping("/check")
public AuthCheckResponse check(
    Authentication authentication
) {
    return new AuthCheckResponse(
        true,
        "jwt-cookie",
        authentication.getName()
    );
}
}

```

```
// PageController.java
@Controller
public class PageController {

    @GetMapping("/")
    public String root() {
        return "redirect:/login.html";
    }

    @GetMapping("/login")
    public String login() {
        return "redirect:/login.html";
    }

    @GetMapping("/signup")
    public String signup() {
        return "redirect:/signup.html";
    }

    @GetMapping("/home")
    public String home() {
        return "redirect:/home.html";
    }
}
```

- Controller 계층은 클라이언트의 HTTP 요청을 가장 먼저 받는 계층
- 기존 코드에서는 createContext를 사용해서 URL마다 직접 요청 처리 코드를 등록함.
- 변경 후에는 각 어노테이션마다 요청 경로와 실행할 메서드를 연결.

2.6. 어노테이션

이름	설명	비고
@SpringBootApplication	Spring Boot 애플리케이션의 시작 클래스에 붙이며, 자동 설정과 컴포넌트 스캔을 활성화한다.	애플리케이션 실행 계열
@Configuration	설정 클래스를 Bean으로 등록할 때 사용한다.	설정 계열
@ConfigurationProperties	application.yml 또는 application.properties의 설정값을 자바 객체에 바인딩한다.	설정 계열
@Component	Spring이 관리하는 일반 Bean으로 등록한다.	Bean 등록 계열
@Service	비즈니스 로직을 담당하는 Service 계층 클래스를 Bean으로 등록한다.	Bean 등록 계열
@Repository	DB 접근을 담당하는 Repository 계층 클래스를 Bean으로 등록한다.	Bean 등록 계열
@RestController	HTTP 요청을 처리하고 메서드 반환값을 JSON 같은 응답 본문으로 반환한다.	Web Controller 계열
@RequestMapping	Controller 클래스나 메서드에 공통 요청 경로를 지정한다.	Web Mapping 계열
@GetMapping	GET 요청을 특정 메서드와 연결한다.	Web Mapping 계열
@PostMapping	POST 요청을 특정 메서드와 연결한다.	Web Mapping 계열
@RestControllerAdvice	여러 RestController에서 발생하는 예외를 공통으로 처리하는 클래스를 등록한다.	예외 처리 계열
@ExceptionHandler	특정 예외가 발생했을 때 실행할 처리 메서드를 지정한다.	예외 처리 계열

위 어노테이션 이외에도 다양한 종류의 어노테이션이 존재함. 그때 그때 AI를 통해 찾아볼 것

2.7. 실행 시점 정리

1. SpringApplication.run이 호출되면서 스프링 부트가 시작
2. application.yml 로드를 통해 환경을 구성.
3. 컴포넌트 스캔 범위는 main이 실행된 클래스의 위치에서 하위 폴더 전부.
4. @Configuration 클래스들이 먼저 등록. 다른 빈(Beans)들이 이 설정에 의존하는 경우가 많아 우선순위가 높음.
5. Spring Security 필터 체인 설정이 등록. 모든 요청은 이 필터를 거침.
6. @Entity 클래스들이 JPA에 의해 인식되어, DB 접속 정보를 바탕으로 테이블과 매핑

7. Spring Data JPA가 Domain의 Entity 정보를 바탕으로 Repository 인터페이스의 구현체를 자동으로 생성.
8. @Service 빈들이 등록되면서 Repository를 주입 받음.
9. Controller Bean들이 등록되고 Service를 주입받으면서, URL과 메서드가 매핑.
10. @ControllerAdvice 등 예외 처리 핸들러가 등록되고, 이후 컨트롤러에서 발생하는 예외를 가로챌 준비를 마칩.
11. 모든 빈이 완성되면 application.yml에 설정된 포트로 내장 웹서버가 열리고, 요청을 받을 준비를 함.
12. 4~9번의 처리가 순서대로 되는 것이 아닌, 스프링이 의존관계 그래프를 보고 필요한 순서대로 빈을 생성.
13. 다만 "설정 -> 보안 -> 도메인/레포 -> 서비스 -> 컨트롤러 -> 예외처리" 이 흐름 자체는 실제 의존 방향과 일치함.

3. 트러블 슈팅

3.1 실행방법

```
./gradlew clean bootJar
: 기존 빌드 결과물을 지우고, 실행 가능한 JAR 파일을 새로 생성. 파일명은, settings.gradle의 프로젝트 이름 + build.gradle의 버전이 합쳐져서 결정됨.
```

```
java -jar build/libs/loginauth-springboot-0.0.1-SNAPSHOT.jar
: 생성된 JAR을 직접 실행합니다.
```

*개발 중 바로 실행은 ./gradlew bootRun 명령어를 통해 테스트.

3.2 오류 발생

배포용 JAR 생성 후 실행했을 때, 테스트가 끝나고 터미널에서 Ctrl+Z를 통해 프로세스를 서스펜드해도 포트번호가 사용 중인 현상을 발견.

원인

- Ctrl+Z는 실행 중인 프로세스를 종료하는 명령이 아니라, 백그라운드에 일시정지시키는 명령이다.
- 따라서 Spring Boot 서버 프로세스가 완전히 종료되지 않고 남아있기 때문에, 기존에 사용하던 포트가 계속 점유된다.
- 이 상태에서 다시 서버를 실행하면 이미 사용 중인 포트라는 오류가 발생한다.

해결 방법

```
jobs
```

- 현재 터미널에서 일시정지된 작업 목록을 확인한다.

```
fg
```

- 일시정지된 작업을 다시 foreground로 가져온다.

```
Ctrl+C
```

- foreground로 가져온 서버 프로세스를 정상 종료한다.

만약 터미널에서 작업 목록을 찾기 어렵다면, 포트를 사용 중인 프로세스를 직접 찾아 종료할 수 있다.

```
lsof -i :8080
```

- 8080 포트를 사용 중인 프로세스의 PID를 확인한다.

```
kill -9 PID
```

- 확인한 PID를 넣어서 해당 프로세스를 강제로 종료한다.

정리

- 서버를 종료할 때는 `Ctrl+Z`가 아니라 `Ctrl+C`를 사용해야 한다.
- `Ctrl+Z`를 이미 눌렀다면 `fg`로 다시 가져온 뒤 `Ctrl+C`로 종료한다.